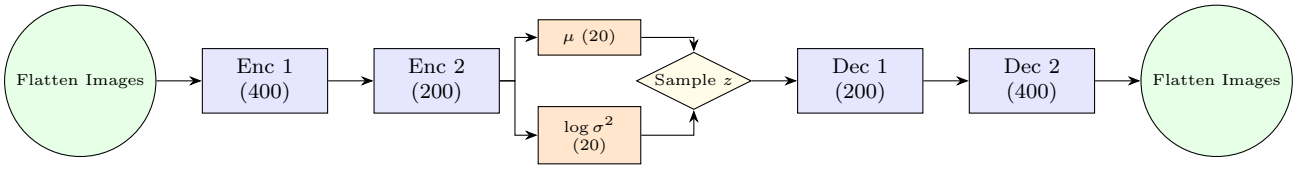
Probabilistic Generative Models Coursework 2

CID: 06046958

March 2026

The background of benchmark VAE

The benchmark model we trained consists two layers in the both encoder and decoder. The basic architecture is demonstrated as:



Additionally, we summary the parameters of our neural network below:

Layer	Type	Input Dimension	Output Dimension
Encoder (Fully Connected)			
Input	-	-	784 (28×28 flattened)
encode1	Linear	784	400
encode2	Linear	400	200
mu_layer	Linear	200	20
logvar_layer	Linear	200	20
Decoder (Fully Connected)			
decode1	Linear	20	200
decode2	Linear	200	400
final	Linear	400	784

Table 1: Summary of MLP-based Benchmark VAE Architecture

In a variational autoencoder, the objective function consists of a reconstruction term and a KL divergence term, which together govern the training of both the encoder and the decoder. The main observations from the loss curves are as follows. First, the training and test losses largely overlap, suggesting that there is no obvious sign of overfitting. Second, the training loss of the benchmark model decreases steadily and approaches convergence within 70 epochs. Meanwhile, there is a clear trade-off between the reconstruction term and the KL divergence: the reconstruction loss decreases as training proceeds, whereas the KL divergence increases. This indicates that the model achieves better reconstruction at the cost of the latent distribution becoming less closely aligned with the prior. Since the overall training loss becomes largely stable after around 60 epochs, we choose 70 epochs as a reasonable stopping point, balancing computational cost against further performance gains.

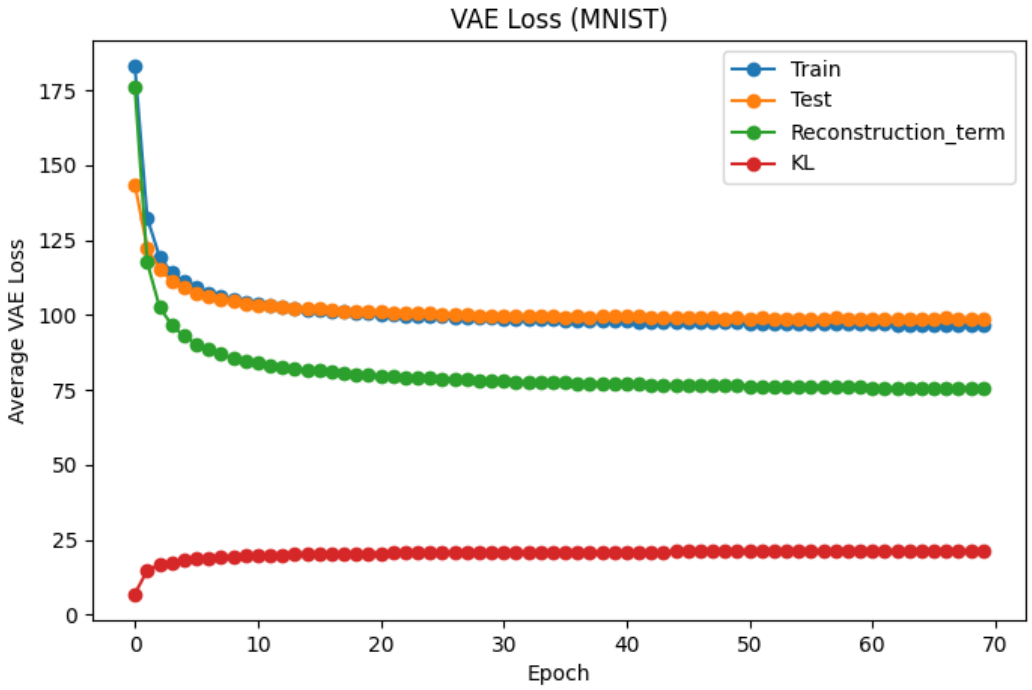


Figure 1: The losses of VAE training

Sampling from prior vs aggregated posterior

There are two ways to generate MNIST samples using this model:

- Generate a standard Gaussian random vector in 20 dimensions and pass it to the decoder.
- By sampling images from the MNIST dataset and passing them through the encoder, we obtain the mean and variance of the corresponding Gaussian latent variable. We then sample from the resulting 20-dimensional Gaussian distribution and feed the samples into the decoder.

However, the samples generated from the prior distribution are of lower quality than those generated from the aggregated posterior, with blurrier shapes and less recognisable digits. This suggests a mismatch between the aggregated posterior and the standard normal prior. In particular, the decoder is mainly trained on latent codes produced by the encoder from real data, so it learns to reconstruct well in regions of latent space supported by the aggregated posterior. If the aggregated posterior does not align closely with the prior, then samples drawn from the prior may fall in regions that are less familiar to the decoder, resulting in poorer generation quality.

Comparison between prior distribution and the aggregated posterior

We study the mean of variance of aggregated posterior as our metrics to compare with the standard normal distribution. By the assumption of VAE, the distribution of latent variable is $q_\theta(z \mid x) = \mathcal{N}(z; \mu_\theta(x), \Sigma_\theta(x))$. Therefore, by Monte Carlo, we could estimate $\mathbb{E}(Z) = \mathbb{E}\{\mathbb{E}(Z \mid X)\}$ by computing $\Sigma_{i=1}^{N} \mu_\theta(x_i)/N$, which will return a vector with 20 dimensions. We

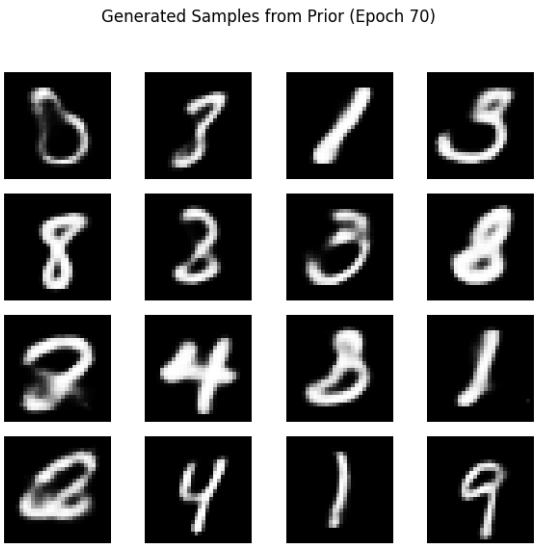


Figure 2: Sampling from prior

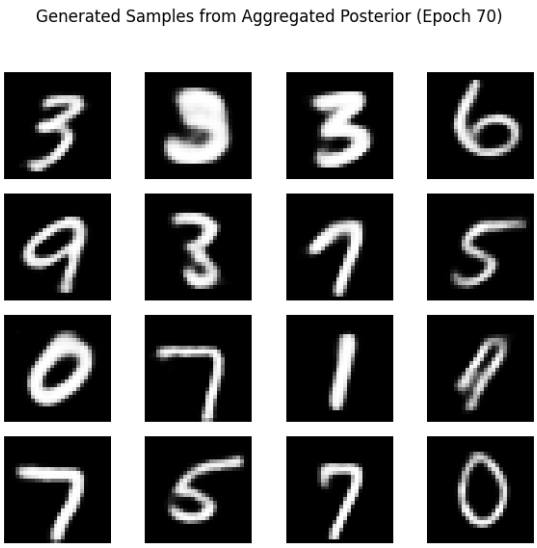


Figure 3: Sampling from aggregated posterior

also consider $\mathrm{Var}(Z) = \mathbb{E}(\mathrm{Var}(Z \mid X)) + \mathrm{Var}(\mathbb{E}(Z \mid X))$, where in each part we could use Monte Carlo to estimate. These are valid as we assume each components in the normal vector are mutually independent. So even though we deal with the vector objects, Monte Carlo provides good estimations. More specifically, we calculate the distance between the $\mathbb{E}(Z)$ and the mean of standard normal and the mean of trace for the variance, which is 0.0028 for distance and 0.9850 for the mean of trace. A low-dimensional projection via t-SNE is also considered, which provides an intuitive visualization. The MNIST dataset can be regarded as a discrete uniform distribution where each sample has the same probability. However, according to the labels assigned to each image, the data can be divided into 10 different digit classes, which resembles a mixture of Gaussian distributions. This observation suggests that the latent representations may form clustered structures corresponding to different digit classes.

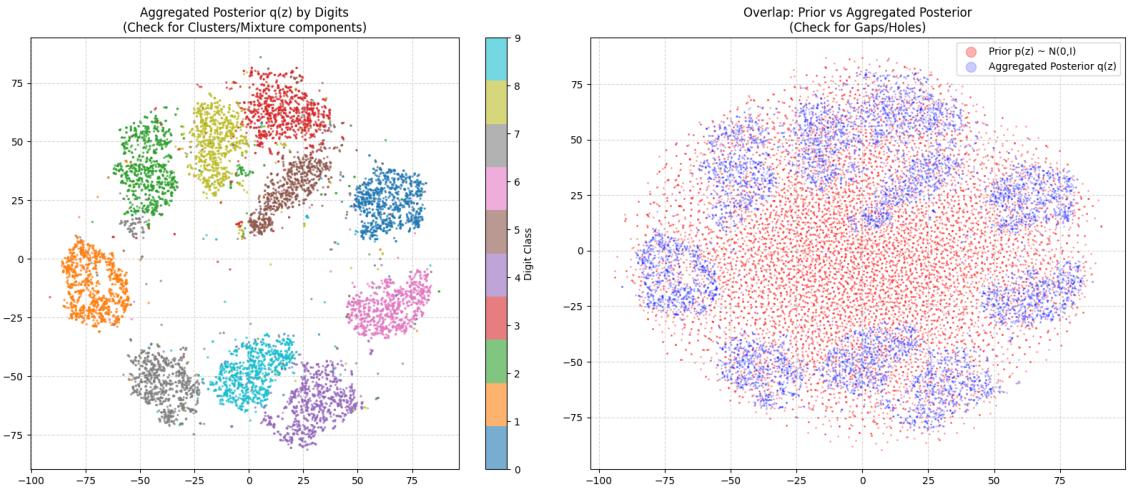


Figure 4: Low dimensional projection

The squared **Wasserstein-2 distance** between the aggregated posterior and the standard

normal prior is 8.1693. Since a value of zero would indicate an exact match, this already suggests a non-negligible discrepancy between the two distributions. This is consistent with the other observations: although the empirical mean is close to zero and the average marginal variance is close to one, the latent representations still exhibit clear class-dependent clustering in the low-dimensional projection. Therefore, the aggregated posterior matches the prior at the level of summary statistics, but differs from it in its overall geometric structure, which mainly constitutes the Wasserstein-2 distance here.

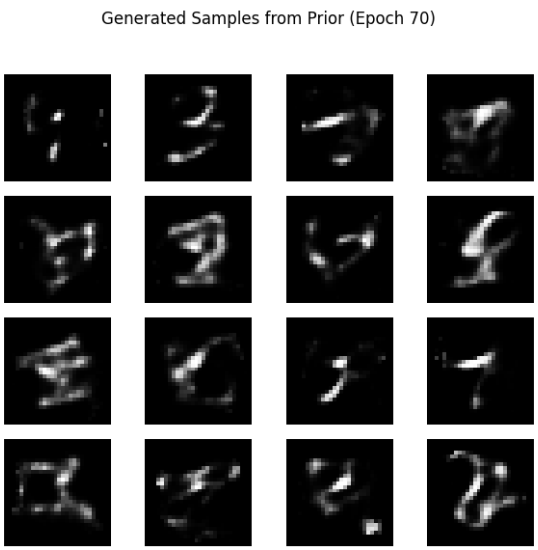
Reconstruction and regularisation

The parameter β controls the trade-off between reconstruction quality and regularisation towards the prior. When β increases, a stronger penalty is imposed on the KL divergence term, forcing the approximate posterior to stay closer to the prior distribution. This usually leads to a more regular latent space, but may reduce the amount of information that the latent code can retain about the input, and hence can worsen reconstruction quality. In contrast, when β decreases, the KL penalty becomes weaker, allowing the latent variables to preserve more information from the data. This often improves reconstruction, but may also lead to a greater mismatch between the aggregated posterior and the prior, which can harm the quality of samples generated directly from the prior. Furthermore, we consider two extreme cases, namely, $\beta \to 0$ and $\beta \to \infty$.

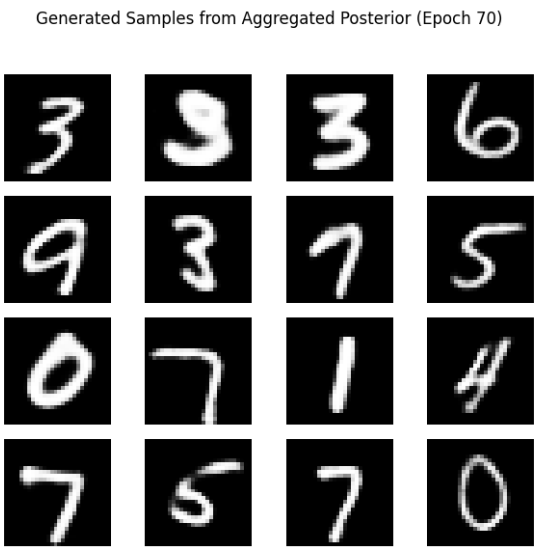
- When $\beta \to 0$, the loss function is dominated by the reconstruction term, meaning the regularization of the latent space structure is unsatisfactory, as indicated by a high Wasserstein-2 distance. Consequently, the decoder cannot effectively recognize signals sampled from the standard normal distribution. This results in the generation of very poor samples, even though the rough shapes of the figures remain observable.

- When $\beta \to \infty$, the loss function is dominated by the KL divergence, meaning the latent space is forced to be extremely close to the standard normal prior. However, the reconstruction term is insufficiently optimized, leading to a degraded decoder. As a result, the decoder loses the ability to distinguish between different classes of figures.

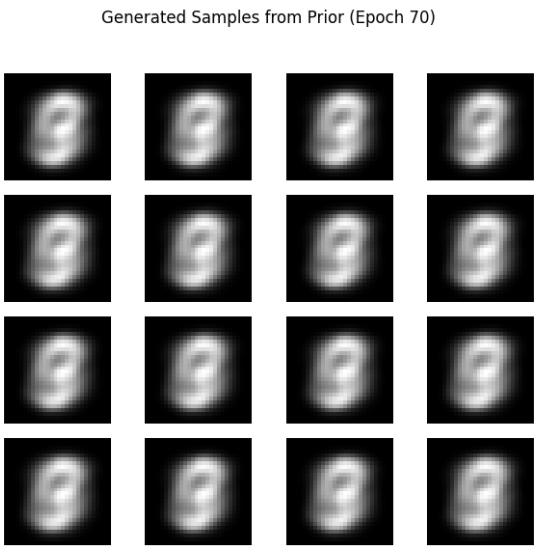
Overall, the experiments show that tuning β mainly changes the trade-off between reconstruction quality and latent regularization, rather than improving all aspects of the model simultaneously. In terms of latent-space alignment, the Wasserstein distance is 1524.65 for $\beta = 0.001$, 614.91 for $\beta = 0.1$, 8.17 for $\beta = 1$, 80.26 for $\beta = 10$, and 1.48×10^{-5} for $\beta = 1000$. These results indicate that very small values of β lead to a substantial mismatch between the aggregated posterior and the prior, whereas an extremely large β forces an almost exact prior match. However, the intermediate cases show that this effect is not monotonic in practice. Combined with the visual comparison of generated samples, these results suggest that extreme values of β push the model towards one side of the trade-off: a very large β enforces strong latent regularization at the cost of poor reconstruction and weak sample quality, whereas a very small β improves reconstruction fidelity but severely degrades samples generated directly from the prior.However, by analysing the effect of both increasing and decreasing β, we find that tuning β may yield slight improvements, but the overall benefit is limited. Therefore, $\beta = 1$ remains the most balanced choice among the values tested.



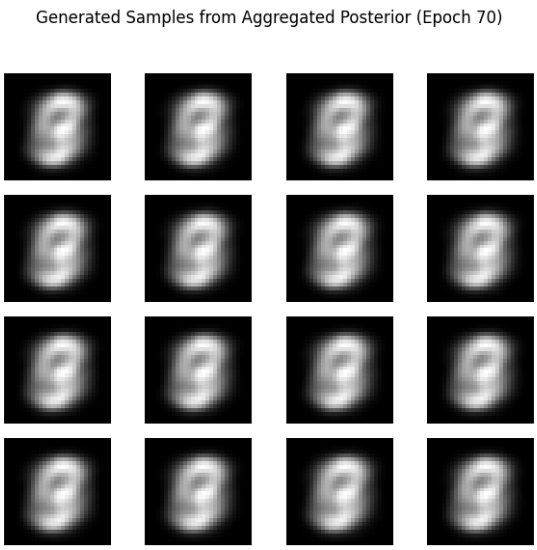
(a) Sampling from prior ($\beta = 0.001$)



(b) Sampling from aggregated post ($\beta = 0.001$)

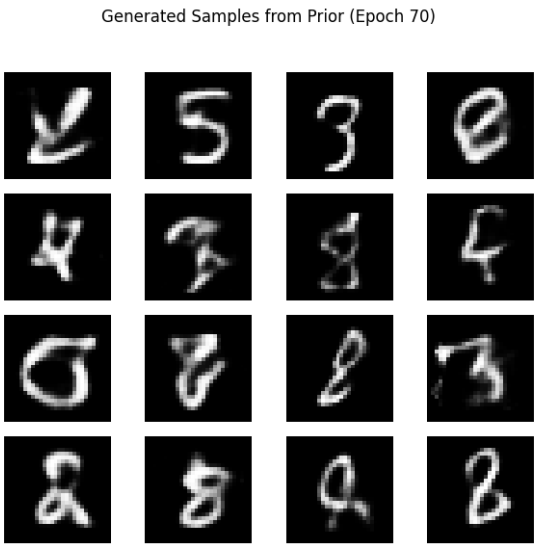


(c) Sampling from prior ($\beta = 1000$)

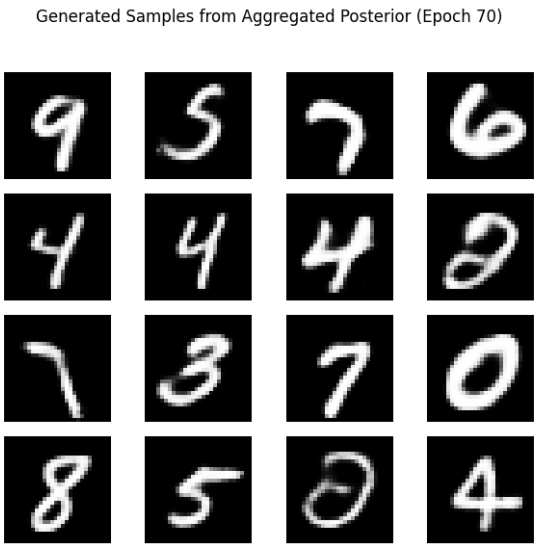


(d) Sampling from aggregated post ($\beta = 1000$)

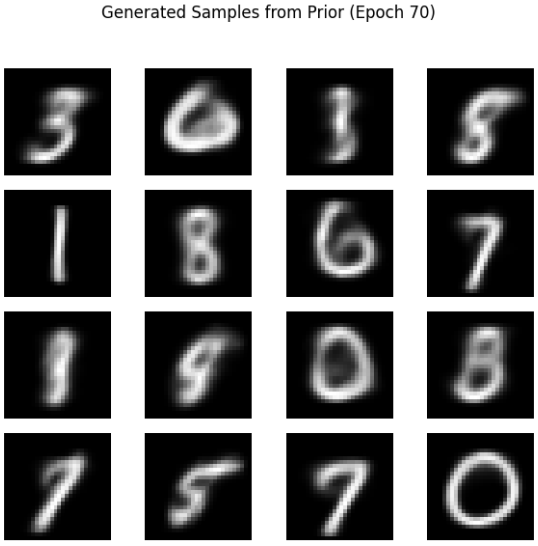
Figure 5: The comparison when β is in the extreme case



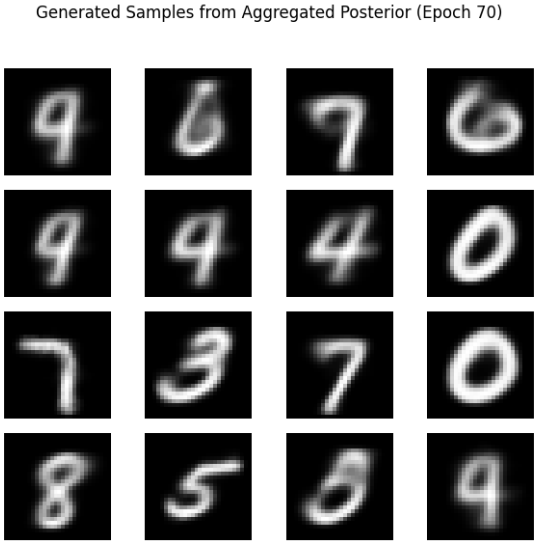
(a) Sampling from prior ($\beta = 0.1$)



(b) Sampling from aggregated post ($\beta = 0.1$)



(c) Sampling from prior ($\beta = 10$)



(d) Sampling from aggregated post ($\beta = 10$)

Figure 6: The comparison when β is in the moderate case

Improved VAE using convolutional neural network

We evaluated the model across different numbers of convolutional layers ($K = 2, 3, 4$) and found that $K = 3$ provides an optimal balance between the quality of generated samples and computational cost. Since all three configurations were trained for the same number of epochs, their total training times remained highly comparable.

Model Depth	Training Time (s)	Mean Dist.	$\mathbb{E}[\sigma^2]$	Wasserstein
2-Layer ConvVAE	753.81	0.0365	0.9798	**8.0450**
3-Layer ConvVAE	769.60	0.0407	0.9852	8.1552
4-Layer ConvVAE	806.01	**0.0102**	**0.9859**	8.1358

Table 2: Trade-off Analysis: Performance vs. Computational Cost across ConvVAE Depths

Layer	Type	In Channels	Out Channels	Kernel/Stride/Pad	Output Shape
Encoder Section					
Input	-	-	-	-	$1 \times 28 \times 28$
conv1	Conv2d	1	16	$3 \times 3, s = 2, p = 1$	$16 \times 14 \times 14$
conv2	Conv2d	16	32	$3 \times 3, s = 2, p = 1$	$32 \times 7 \times 7$
conv3	Conv2d	32	64	$3 \times 3, s = 2, p = 1$	$64 \times 4 \times 4$
fc_encode	Linear	-	-	In: 1024, Out: 128	128
fc_mu/var	Linear	-	-	In: 128, Out: 20	20
Decoder Section					
fc_decode	Linear	-	-	In: 20, Out: 128	128
fc_upsample	Linear	-	-	In: 128, Out: 1024	$64 \times 4 \times 4$
deconv0	ConvTr2d	64	32	$3 \times 3, s = 2, p = 1, op = 0$	$32 \times 7 \times 7$
deconv1	ConvTr2d	32	16	$3 \times 3, s = 2, p = 1, op = 1$	$16 \times 14 \times 14$
deconv2	ConvTr2d	16	1	$3 \times 3, s = 2, p = 1, op = 1$	$1 \times 28 \times 28$

Table 3: Summary of 3-Layer ConvVAE Architecture Parameters

We then compared the quality of generated images:

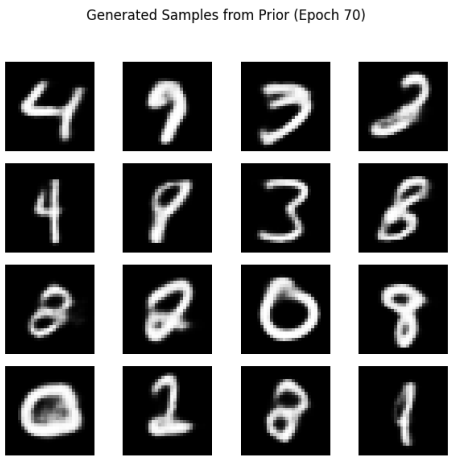


Figure 7: Sampling from prior

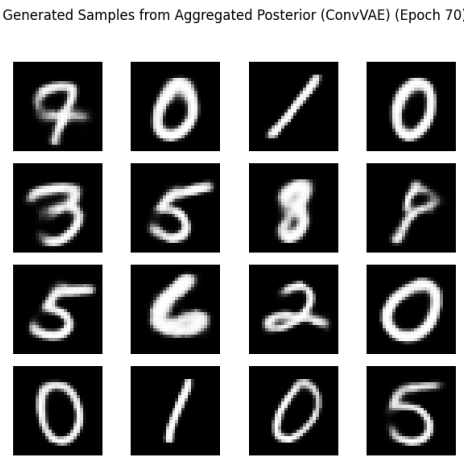


Figure 8: Sampling from agg post

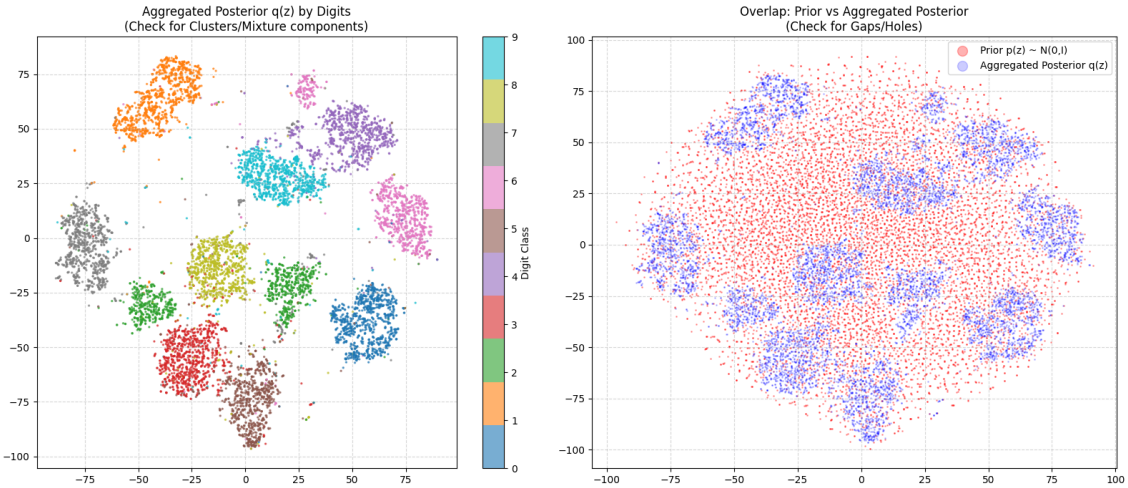


Figure 9: Low dimensional projection

Comments: From Figure 7 and 8, it is evident that the samples generated from the prior distribution closely resemble those from the aggregated posterior. Both sets of images exhibit distinct digit shapes with minimal blurriness. This similarity demonstrates that the latent space is well-structured, allowing the prior to serve as an effective generative proxy for the learned data distribution. In the latent space, the aggregated posterior still does not perfectly match the standard normal prior, since the low-dimensional projection shows clear class-dependent clustering. In addition to this, the visual quality of generated samples is improved, which indicates the convolutional architecture provides a more suitable inductive bias for image data.

We replicated the experiment to evaluate the aforementioned metrics and conduct a comparative analysis against the benchmark model.

Metric	**ConvVAE (CNN)**	**Benchmark (MLP)**	**Ideal**
Mean Distance	0.0365	0.0028	0
Avg. Posterior Var ($\mathbb{E}[\sigma^2]$)	0.9798	0.9850	1
Wasserstein Distance	8.0450	8.1693	0

Table 4: Comparison of Latent Space Metrics between ConvVAE and Benchmark Model

Table 4 compares the distance of the posterior mean from the origin, the average posterior variance, and the Wasserstein-2 distance for the ConvVAE and the benchmark MLP-based VAE. The two models exhibit very similar latent-space summary statistics. In particular, the Mean Distance is slightly larger for the ConvVAE (0.0365 versus 0.0028), while the Average Posterior Variance is slightly lower (0.9798 versus 0.9850). The Wasserstein distance is also marginally lower for the ConvVAE (8.0450 versus 8.1693), suggesting a slightly closer match between the aggregated posterior and the standard normal prior. Overall, these numerical differences are small, indicating that the two models have broadly similar latent-space distributions at the level of these summary metrics. However, the visual results show that the ConvVAE generates noticeably sharper and more coherent digits. This suggests that latent-space summary statistics alone do not fully explain generative performance, and that the convolutional architecture improves decoding quality by better capturing the spatial structure of image data. Therefore, even when the latent-space metrics are only marginally different, the ConvVAE achieves a clear qualitative improvement over the benchmark model.

From the above analysis of the evaluation metrics and the quality of the generated images, we conclude that CNN-based architectures improve the overall performance of the VAE model. CNNs are more capable of capturing the correlations between neighbouring pixels through convolutional filters. In the context of the MNIST dataset, where images have a two-dimensional spatial structure, convolutional layers can effectively exploit local patterns such as edges and strokes. This allows the encoder to learn more informative latent representations and enables the decoder to reconstruct images with clearer and more realistic digit structures. This observation highlights an important limitation of fully-connected VAEs: they fail to exploit the spatial inductive bias present in image data, while convolutional architectures enable the latent representation to capture more meaningful generative factors.

MATH70148 - Probabilistic Generative Models 2025-2026

Coursework 2 - Part II

Instructions:

You will implement a minimal version of the GPT architecture and train it to generate synthetic text in the style of the Shakespeare's works. Follow this notebook and complete the missing parts according to the given instructions. Register the code you generate in the parts indicated as `YOUR CODE HERE` and your responses (when applicable) by creating a new cell in this notebook. Your submission will be assessed from this notebook only, **no pdf report is needed**.

Comment on running time: It is expected that this model is to be trained over at least 5000 epochs to produce text resembling the stucture of a play, and you are strongly encouraged to run it on Google Colab (where the 5000 epochs should take around 6 minutes if using a T4 runtime type) or a computer with a sufficiently capable GPU. For reference, if ran on a regular MPS-enabled Apple M3 Pro Macbook Pro, this code will take ~10 minutes, and ~5.5 minutes if using a NVIDIA GeForce GTX 1080 GPU.

```
import torch
import torch.nn as nn
from torch.nn import functional as F
```

1. Download the corpus and tokenise it

```
# If using Colab, consider:
from google.colab import drive
drive.mount('/content/drive/')
```

```
Drive already mounted at /content/drive/; to attempt to forcibly remount, call drive.mount("/content/drive/", for
```

```
# Load the provided dataset
path = '/content/drive/MyDrive/Imperial College London/PGM/CW2/shakespeare.txt'
```

```
# Read the entire Shakespeare corpus into memory
# We explicitly set UTF-8 encoding to avoid platform-dependent issues.
with open(path, 'r', encoding='utf-8') as file:
    text = file.read()

# Basic corpus statistics
num_chars = len(text)

print("=== Corpus Statistics ===")
print(f"Total number of characters: {num_chars:,}")
print(f"First 200 characters preview:\n")
print(text[:200])
```

```
=== Corpus Statistics ===
Total number of characters: 1,115,394
First 200 characters preview:

First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you
```

```
# Extract the set of unique characters appearing in the text
# `set(text)` removes duplicates
# `sorted(...)` ensures a deterministic ordering (important for reproducibility)
chars = sorted(list(set(text)))

# Size of the vocabulary (number of unique characters)
vocab_size = len(chars)
```

```
# Display results
print("Unique characters (sorted):")
print("".join(chars))  # Join list into a readable string

print("\nVocabulary size:")
print(f"{vocab_size} unique characters")
```

```
Unique characters (sorted):

 !$&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

Vocabulary size:
65 unique characters
```

```
# ----------------------------------------------------
# Character-level tokenizer
# ----------------------------------------------------

# NOTE:
# The model will operate at the character level, meaning that the tokens are characters (and not words)

# Create a mapping from character to integer index
# This is our "string-to-index" (stoi) dictionary.
stoi = {char: idx for idx, char in enumerate(chars)}

# Create the inverse mapping: index to character
# This is needed to convert model outputs back to text.
itos = {idx: char for idx, char in enumerate(chars)}


def encode(s):
    """
    Convert a string into a list of integer token IDs.

    Args:
        s (str): Input string.

    Returns:
        List[int]: List of corresponding character indices.
    """
    return [stoi[char] for char in s]


def decode(indices):
    """
    Convert a list of integer token IDs back into a string.

    Args:
        indices (List[int]): List of character indices.

    Returns:
        str: Decoded string.
    """
    return ''.join(itos[idx] for idx in indices)
```

```
# Convert the entire corpus into a PyTorch tensor of token IDs.
# We use dtype=torch.long because embedding layers expect integer (int64) indices.
data = torch.tensor(encode(text), dtype=torch.long)

# Inspect the shape of the resulting tensor
print("Data tensor shape:", data.shape)
```

```
Data tensor shape: torch.Size([1115394])
```

```
# Display a short excerpt of the original text and its corresponding
# integer (token ID) representation to illustrate the encoding process.

N = 50

print(f"Text containing the first {N} characters:")
print("-" * 50)
print(text[:N])

print("\n" + "-" * 50)
print("Its representation as a PyTorch tensor:\n")
print(data[:N])
```

```
Text containing the first 50 characters:
--------------------------------------------------
First Citizen:
Before we proceed any further, hear

--------------------------------------------------
Its representation as a PyTorch tensor:

tensor([18, 47, 56, 57, 58,  1, 15, 47, 58, 47, 64, 43, 52, 10,  0, 14, 43, 44,
        53, 56, 43,  1, 61, 43,  1, 54, 56, 53, 41, 43, 43, 42,  1, 39, 52, 63,
         1, 44, 59, 56, 58, 46, 43, 56,  6,  1, 46, 43, 39, 56])
```

2. Dataset split

```
# Split the dataset into training (90%) and validation (10%) sets.
# We consider a sequential split to preserve temporal structure.

n = int(0.9 * len(data))  # 90% for training
train_data = data[:n]
val_data = data[n:]

print(f"Training corpus size: {len(train_data):,} characters "
      f"({train_data.shape[0] / data.shape[0]:.2%} of total)")

print(f"Validation corpus size: {len(val_data):,} characters "
      f"({val_data.shape[0] / data.shape[0]:.2%} of total)")
```

```
Training corpus size: 1,003,854 characters (90.00% of total)
Validation corpus size: 111,540 characters (10.00% of total)
```

On the data–label mechanics

- We access the data through *context windows*: contiguous blocks of text in numerical (token) representation of size `block_size`.
- The model is self-supervised. That is, we train it so that, given a sequence of tokens $x_{i:j}$, it predicts the next token x_{j+1}.
- The labels are obtained from the same context window by shifting it with an offset of 1 position.

```
# EXAMPLE:

# Given a context window of size 7,
# we construct an input–target pair (X, Y),
# where Y is simply X shifted by one position.

block_size = 7

print("A context window (X, Y) looks like:\n")

# Input sequence (context)
X = data[:block_size]
print(f"X (token IDs): {[x.item() for x in X]}")
print(f"--> decode(X): {decode([x.item() for x in X])}")

print("-" * 40)

# Target sequence (shifted by one position)
Y = data[1:block_size + 1]
print(f"Y (token IDs): {[y.item() for y in Y]}")
print(f"--> decode(Y): {decode([y.item() for y in Y])}")
```

```
A context window (X, Y) looks like:

X (token IDs): [18, 47, 56, 57, 58, 1, 15]
--> decode(X): First C
----------------------------------------
Y (token IDs): [47, 56, 57, 58, 1, 15, 47]
--> decode(Y): irst Ci
```

```
# EXAMPLE (continued):

# However, within each context window we use the information
# in an autoregressive manner, generating multiple training
# examples from a single block.

x = train_data[:block_size]
y = train_data[1:block_size + 1]
```

```
for t in range(block_size):
    context = x[:t + 1]      # Tokens available at time step t
    target = y[t]            # Next token to predict

    print(f"When the input is {context.tolist()}, the target is: {target.item()}")
```

```
When the input is [18], the target is: 47
When the input is [18, 47], the target is: 56
When the input is [18, 47, 56], the target is: 57
When the input is [18, 47, 56, 57], the target is: 58
When the input is [18, 47, 56, 57, 58], the target is: 1
When the input is [18, 47, 56, 57, 58, 1], the target is: 15
When the input is [18, 47, 56, 57, 58, 1, 15], the target is: 47
```

Therefore, each context window generates a number of training observations equal to its length.

We now turn to *batches*, which allows us to process multiple context windows in parallel. The key point is that each context window is independent: there is no computation that occurs across windows, only parallel computation within them. In other words, the autoregressive sequences corresponding to different context windows are not mixed. See the following example.

```
torch.manual_seed(1337)

batch_size = 4
block_size = 8  # maximum context length considered for prediction
# These hyperparameters will be redefined for the final training phase


def get_batch(split):
    """
    Generate a small batch of input (x) and target (y) sequences.

    Each batch contains `batch_size` independent context windows
    of length `block_size`, sampled randomly from the dataset.
    """
    data_split = train_data if split == 'train' else val_data

    # Random starting positions for each context window
    ix = torch.randint(len(data_split) - block_size, (batch_size,))

    # Construct input and target batches
    x = torch.stack([data_split[i:i + block_size] for i in ix])
    y = torch.stack([data_split[i + 1:i + block_size + 1] for i in ix])

    return x, y


# Sample a training batch
xb, yb = get_batch('train')

print("Inputs:")
print("Shape:", xb.shape)   # (batch_size, block_size)
print(xb)

print("\nTargets:")
print("Shape:", yb.shape)   # (batch_size, block_size)
print(yb)
```

```
Inputs:
Shape: torch.Size([4, 8])
tensor([[24, 43, 58,  5, 57,  1, 46, 43],
        [44, 53, 56,  1, 58, 46, 39, 58],
        [52, 58,  1, 58, 46, 39, 58,  1],
        [25, 17, 27, 10,  0, 21,  1, 54]])

Targets:
Shape: torch.Size([4, 8])
tensor([[43, 58,  5, 57,  1, 46, 43, 39],
        [53, 56,  1, 58, 46, 39, 58,  1],
        [58,  1, 58, 46, 39, 58,  1, 46],
        [17, 27, 10,  0, 21,  1, 54, 39]])
```

Questions

- Q1 (5% marks): Explain why autoregressive language modeling is considered self-supervised. What are the inputs and labels in this setup?
- Q2 (5% marks): From a context window of size T, how many training examples are generated? Explain why.

Q1: Autoregressive language modeling is considered self-supervised because it extracts supervision signals directly from the raw data, reducing the need for manual human labeling in each input text. In this setup, the inputs are defined as a sequence of preceding tokens, $(x_1, x_2, \dots, x_n)$, while the label is the immediate subsequent token, x_{n+1}, which serves as the ground truth. During training, the model generates a probabilistic prediction for the next element. The discrepancy between this prediction and the true token constitutes a loss (typically cross-entropy), providing the necessary signal to update the model's parameters.

Q2: For a single sequence $(x_1, x_2, \dots x_T)$with size T, we may have T training samples. As it is an autogression model, we may start predicting x_2 given x_1, and finally predict x_{T+1} given $x_1, x_2, \dots x_T$.

3. Self-attention

- Q3 (15% marks): Implement the missing parts in the attention head class. Recall that in this model you need to implement masked attention.

```
# Embedding dimension (model "width"): each token is represented as a vector in R^{n_embd}
n_embd = 64
dropout = 0.0


class Head(nn.Module):
    """
    A single self-attention head (causal / autoregressive).

    Given an input sequence of token embeddings x of shape (B, T, n_embd),
    this module produces an output of shape (B, T, head_size) by:
      1) projecting x into queries, keys, and values
      2) computing attention weights via scaled dot-product attention
      3) applying a causal mask so tokens cannot attend to the future
      4) computing a weighted sum of value vectors
    """

    def __init__(self, head_size: int):
        super().__init__()


        # ------------------------------------------------------------ #
        # --------------------[YOUR CODE HERE]--------------------- #
        # ------------------------------------------------------------ #
        self.key = nn.Linear(n_embd, head_size, bias=False) #(n_embd, dk)
        self.query = nn.Linear(n_embd, head_size, bias=False) #(n_embd, dk)
        self.value = nn.Linear(n_embd, head_size, bias=False) # (n_embd, dv)


        # Causal mask (lower-triangular): prevents attending to future positions.
        # register_buffer makes it part of the module state (moves with .to(device))
        # but it is not a learnable parameter.
        self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))

        # Dropout applied to the attention weights (common regularization in Transformers)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:


        """
        Args:
            x: (B, T, n_embd) input token embeddings

        Returns:
            out: (B, T, head_size) attention output for this head
        """
        T = x.shape[1]

        # ------------------------------------------------------------ #
        # --------------------[YOUR CODE HERE]--------------------- #
        # ------------------------------------------------------------ #
        k = self.key(x)   # (B, T, dk) and key is a network (matrix)
        q = self.query(x) # (B, T, dk) and query is a network (matrix)
        v = self.value(x) # (B, T, dv) and value is a network (matrix)

        # Compute attention weights
        weights = q @ k.transpose(-2, -1) * (k.shape[-1]**-0.5) # (B, T, dk) @ (B, dk, T) -> (B, T, T)
```

```
        # Apply causal mask (prevents tokens from attending to future tokens)
        weights = weights.masked_fill(self.tril[:T, :T] == 0, float('-inf'))

        # Softmax to get attention probabilities
        weights = F.softmax(weights, dim=-1)

        # Apply dropout to attention weights
        weights = self.dropout(weights)

        # Perform the weighted aggregation of the values
        out = weights @ v  # (B, T, T) @ (B, T, dv) -> (B, T, dv)

        return out
```

Questions

- Q3 (continued): Explain why the mask is needed.

In autoregressive language modeling, a causal mask is necessary to prevent 'information leakage.' Since the entire sequence is available during training, without a mask, the model would be able to 'cheat' by directly attending to future tokens that it is supposed to predict. To prevent this, the mask is applied to the attention scores (the dot product of current queries and future keys) by setting them to negative infinity ($-\infty$). This ensures that after the softmax operation, the attention weights for future positions become zero, forcing the model to rely solely on the preceding context to make its predictions.

4. Multi-head Attention

- Q4 (10% marks): Implement the missing parts in the multihead attention class.

```
class MultiHeadAttention(nn.Module):
    """
    Multiple self-attention heads running in parallel.

    Each head independently computes attention over the same input.
    Their outputs are concatenated and linearly projected back to the
    embedding dimension (n_embd).
    """

    def __init__(self, num_heads: int, head_size: int):
        super().__init__()

        # ------------------------------------------------------------ #
        # --------------------[YOUR CODE HERE]-------------------- #
        # ------------------------------------------------------------ #
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)]) #(num_head, n_embd * hea
        self.proj = nn.Linear(head_size * num_heads, n_embd)  # (head_size * num_head, n_embd)

        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (B, T, n_embd)

        Returns:
            out: (B, T, n_embd)
        """
        # ------------------------------------------------------------ #
        # --------------------[YOUR CODE HERE]-------------------- #
        # ------------------------------------------------------------ #
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.proj(out)
        out = self.dropout(out)

        return out
```

5. MLP stage

- Q5 (7.5% marks): Implement the missing parts in the FeedForward attention class.

```
class FeedForward(nn.Module):
    """
    Position-wise Feed-Forward Network (single-hidden layer MLP)
    This module is applied independently to each position in the sequence.

    Input:  (B, T, n_embd)
    Output: (B, T, n_embd)
    """

    def __init__(self, n_embd: int):
        super().__init__()

        # The Transformer FFN consists of two linear layers with a non-linearity in between.
        # The inner layer expands the dimensionality by a factor of 4,
        # then projects back to the original embedding size.

        # ------------------------------------------------------- #
        # --------------------[YOUR CODE HERE]-------------------- #
        # ------------------------------------------------------- #
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd)
        )



    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Applies the feed-forward network independently at each time step.
        No interaction occurs across sequence positions.
        """
        return self.net(x)
```

Questions

- Q6 (10% marks): Explain the relationship between the hyperparameters `n_head` and `head_size` in the `MultiHeadAttention` class. Consider their role within the decoder block (i.e., attention + feed-forward).

We state the relationship is `n_embd = n_head * head_size` which defines the decomposition of the embedding space into multiple parallel subspaces. This allows the multi-head attention mechanism to process tokens in 'batches' of dimensions, capturing diverse linguistic aspects simultaneously. Within the decoder block, this identity ensures that the attention output maintains the original `n_embd` dimension, which is essential for residual connections and for providing a compatible input to the subsequent feed-forward layer.

6. Decoder block

- Q7 (10% marks): Implement the missing parts in the DecoderBlock class.

```
class DecoderBlock(nn.Module):
    """
    Transformer decoder block: COMMUNICATION followed by COMPUTATION.

    - Communication: masked multi-head self-attention (tokens exchange information)
    - Computation: position-wise feed-forward network (nonlinear transformation per token)

    Your implementation should use the Pre-LayerNorm (Pre-LN) variant:
    LayerNorm is applied before each sublayer, and residual (skip) connections are applied around each sub
    """

    def __init__(self, n_embd: int, n_head: int):
        # n_embd: embedding dimension, n_head: number of attention heads

        super().__init__()

        # ------------------------------------------------------- #
        # --------------------[YOUR CODE HERE]-------------------- #
        # ------------------------------------------------------- #

        self.mha = MultiHeadAttention(n_head, n_embd // n_head)
        self.ff = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)
```

```
        def forward(self, x: torch.Tensor) -> torch.Tensor:
            """
            Args:
                x: (B, T, n_embd) input embeddings

            Returns:
                x: (B, T, n_embd) output after attention + feed-forward, with residuals
            """
            # ------------------------------------------------------- #
            # --------------------[YOUR CODE HERE]-------------------- #
            # ------------------------------------------------------- #
            x = x + self.mha(self.ln1(x))
            x = x + self.ff(self.ln2(x))

            return x
```

7. GPT Model: Putting it all together

- Q8 (17.5% marks): Implement the missing computation of the loss in the `GPTLanguageModel` class and the `generate` method to sample tokens that autoregressively extend a sequence.

```
class GPTLanguageModel(nn.Module):
    """
    A minimal GPT-style (decoder-only) Transformer language model.

    Components:
      - Token embeddings: map token IDs -> vectors in R^{n_embd}
      - Positional embeddings: encode token position within the context window
      - A stack of decoder blocks: masked self-attention + feed-forward (+ residuals)
      - Final layer norm + linear head: produce logits over the vocabulary

    Input:
      idx: (B, T) integer token IDs

    Output:
      logits: (B, T, vocab_size)
      loss: scalar cross-entropy loss (if targets are provided)
    """

    def __init__(self):
        super().__init__()

        # Token and position embeddings
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)

        # Stack of decoder blocks
        self.blocks = nn.Sequential(
            *[DecoderBlock(n_embd, n_head=n_head) for _ in range(n_layer)]
        )

        # Final normalization and language modeling head
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # Weight initialization (close to the GPT/Transformer convention)
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx: torch.Tensor, targets: torch.Tensor | None = None):
        """
        Args:
            idx: (B, T) token IDs
            targets: (B, T) token IDs (optional)
```

```
        Returns:
            logits: (B, T, vocab_size)
            loss: scalar loss if targets is provided, else None
        """
        B, T = idx.shape

        # Token embeddings: (B, T, n_embd)
        tok_emb = self.token_embedding_table(idx)

        # Positional embeddings: (T, n_embd)
        # We create positions [0, 1, ..., T-1] on the correct device.
        pos = torch.arange(T, device=idx.device)
        pos_emb = self.position_embedding_table(pos)

        # Add token and positional information (broadcast pos_emb over batch dimension)
        x = tok_emb + pos_emb  # (B, T, n_embd)

        # Decoder blocks + final layer norm
        x = self.blocks(x)     # (B, T, n_embd)
        x = self.ln_f(x)       # (B, T, n_embd)

        # Convert to vocabulary logits
        logits = self.lm_head(x)  # (B, T, vocab_size)

        # ------------------------------------------------------------ #
        # ----------------------[YOUR CODE HERE]---------------------- #
        # --------(computation of the cross entropy loss)--------- #
        loss = None
        if targets is not None:
            # Reshape logits to (B*T, vocab_size) and targets to (B*T)
            # This is necessary for F.cross_entropy which expects (N, C) and (N) respectively
            B, T, C = logits.shape
            logits_flat = logits.view(B * T, C)
            targets_flat = targets.view(B * T)
            loss = F.cross_entropy(logits_flat, targets_flat)

        return logits, loss

    @torch.no_grad()
    def generate(self, idx: torch.Tensor, max_new_tokens: int, temperature: float = 1.0):
        """
        Autoregressively sample new tokens.

        Args:
            idx: (B, T) current context (token IDs)
            max_new_tokens: number of tokens to generate
            temperature: > 0, controls randomness (lower is more deterministic)

        Returns:
            idx: (B, T + max_new_tokens) with generated tokens appended
        """
        self.eval()

        for _ in range(max_new_tokens):

            # ------------------------------------------------------------ #
            # ----------------------[YOUR CODE HERE]---------------------- #
            # ------------------------------------------------------------ #

            # Crop idx to the last block_size tokens
            # This is to ensure we don't feed an infinitely growing context to the model
            idx_cond = idx[:, -block_size:]

            # Get predictions (logits) for the next token
            logits, _ = self(idx_cond)  # (B, T_cond, vocab_size)

            # Focus only on the last time step (the prediction for the next token)
            logits = logits[:, -1, :]  # (B, vocab_size)

            # Apply temperature to control randomness
            logits = logits / temperature

            # Apply softmax to get probabilities
            probs = F.softmax(logits, dim=-1)  # (B, vocab_size)

            # Sample the next token from the distribution
            idx_next = torch.multinomial(probs, num_samples=1)  # (B, 1)

            # Append the sampled token to the current sequence
```

```
            # Append the sampled token to the current sequence
            idx = torch.cat((idx, idx_next), dim=1)

        return idx
```

8. Training specs

↳ 已隐藏 7 个单元格

9. Train

↳ 已隐藏 1 个单元格

10. Generate text sequences with the trained model

```
# ------------------------------------------------------------
# Generate text using the trained model
# ------------------------------------------------------------

# Start from a minimal context (single token with ID 0)
context = torch.zeros((1, 1), dtype=torch.long, device=device)

# Generate 1500 new tokens autoregressively
generated_tokens = model.generate(context, max_new_tokens=1500)

# Decode token IDs back into characters
print(decode(generated_tokens[0].tolist()))


# ------------------------------------------------------------
# Optionally, write generated text to a file
# ------------------------------------------------------------

# open('output.txt', 'w').write(
#     decode(model.generate(context, max_new_tokens=10000)[0].tolist())
# )
```

```
Life:
though in the trouble born an eaged, not that dales
Shall have thee that veing maid
from let in the provid your mother, but one I
On the one the deliver, you marched am here well.
Speak how come on your green?

Lord:
I pray him, I am a very captain and good view;
For I say, if grither we have been, is better.

LEONTES:
Of my wife
The contrain, 'But's to the writer reign vile to thee?

ABHOS:
But that have worm what now hasty that my pardon
Here lies and therefore way; therefore and not the trun.

HENRY BOLINGBROKE:
But, my lord, it was they brother metitlement,
By Knowing of the grooners, uncle, and to thief
Like oft the transman-carr'd, we will smild here with
strong-factify, and doing thee avoid, ask, And
That state as not such a life
For more and shall confirm soul are for love,
Her more die on himself from his captain.

Second Senator:
My Lord, decry Captua,
What they do being butting on my worship
And when two he with his copesity or stream;
But I shall mark thee weakness look named to die.
Hark, Dessing! To tell me villains my proud,
And will I much my ensure.

Second Gentleman:
What do you have my grace, and your imprisonment:
Therefore is multing it.

DUKE VINCENTIO:
```

```
Pray, well, what'st two devised; for your rawesty
Your queen, still drunk ep ye good partivalent;
Here I have no soldiers to be
Be pulck on't. Music
LADY ANNE:

Good GELO:
O, go we'll come at the king,
Tenter'd speech or me and provoked with bitter of me;
Going a morrow to pardon me with parts
To de
```

Questions

- Q9 (20% marks): Comment on the generated text.
  - Does it learn local syntax?
  - Does it maintain long-term coherence?
  - How does the temperature affect the generated text?
  - What architectural limitation explains its behavior?

Yes, the model learns local syntax. It understands how words should be ordered on a surface level. For example, it knows that a verb usually follows a subject and a noun often follows an adjective. This creates the illusion of structure (Subject-Verb-Object), even though the meaning doesn't make sense.

No, the model fails to maintain long-term coherence. While it effectively captures local dependencies (the relationship between neighboring tokens), it struggles to integrate global context from the entire text. This limitation prevents the model from generating a logically consistent narrative, as it prioritizes immediate statistical patterns over long-range semantic structure

In terms of Temperature (T), a higher value increases the model's output stochasticity by flattening the logit distribution. Since logits are scaled as `logits / T` before being passed to the Softmax function, a high T reduces the relative gap between the largest and smallest values. This results in a more uniform (non-peaked) probability distribution, preventing the model from becoming overly confident in a single 'top' token and allowing for more diverse and creative predictions during the sampling process.

The restricted dimension of our embedding layer significantly impacts the model's performance. Compared to GPT-3's 12,288-dimensional embeddings, our model utilizes only 384 dimensions, which severely limits its capacity to capture nuanced textual characteristics and represent complex information. Furthermore, the insufficient number of attention heads prevents the model from effectively resolving syntactic and semantic dependencies, such as distinguishing between subjects and objects, leading to the logical inconsistencies observed in the generated text.